

Accesso Dati in .NET con ADO.NET ed Entity Framework Core

1. Introduzione e Presentazione

- Speaker: Massimiliano Sorce
- Senior Developer @ MSC Technology Italy
- 1 anno in MSC, 20+ anni di esperienza su Microsoft Technology Stack
- Email: massimiliano.sorce@msc.com

2. Da ADO.NET a Entity Framework Core

2.1. Che Cosa è ADO.NET

- Definizione: Insieme di classi per servizi di accesso ai dati per programmatori .NET Framework.

Componenti principali:

- **.NET Framework Data Providers:** Componenti per manipolazione dati e accesso rapido, forward-only, sola lettura.
- **Connection:** Connettività all'origine dati.
- **Command:** Accesso a comandi database (restituire/modificare dati, stored procedure).
- **DataReader:** Flusso di dati ad alte prestazioni (forward-only, read-only).
- **DataAdapter:** Ponte tra DataSet e origine dati, usa Command per SQL.

- **DataSet:** Cache locale dei dati, indipendente dall'origine dati, può contenere DataTable con righe, colonne, chiavi e relazioni.

2.2. Modelli di Connessione ADO.NET

- ADO.NET: Fornisce accesso coerente a origini dati (SQL Server, XML, OLE DB, ODBC).
- Due modelli principali:
- Modello Connesso:
- Implementato tramite classi come SqlConnection e SqlDataReader.
- SqlDataReader: Veloce ed efficiente per leggere dati, ma connessione aperta per tutta la durata della lettura.
- Opera in modalità forward-only e read-only.
- Problema: Oneroso per scalabilità e gestione risorse (connessioni aperte).
- Problema: Materializzazione o mappatura manuale da tipi generici (stringhe/objects) a tipi .NET specifici (int, DateTime).
- Conseguenza interruzione connessione: Eccezione.
- Modello Disconnesso:
- Rappresentato dalla classe DataSet.
- Soluzione storica: Tentativo di risolvere il problema delle connessioni aperte del DataReader.

- DataSet: Copia cache locale e completa di una parte del database. Permette di lavorare offline.
- Processo: Riempimento iniziale (chiusura immediata connessione), modifiche in memoria, riapertura connessione per batch update.
- Problema: Molto pesante (usava XML per serializzazione), fortemente tipizzato in modo inflessibile, richiedeva ancora mappatura manuale.

2.3. ADO.NET: Quando Usare i Modelli

- Architettura Connessa:
- Accesso ai dati in tempo reale.
- Coerenza istantanea dei dati.
- Applicazioni come dashboard live o sistemi di transazione.
- Architettura Disconnessa:
- Funzionamento offline o scenari di elaborazione batch.
- Performance e scalabilità essenziali (non si possono mantenere connessioni aperte).
- Lavoro con set di dati di grandi dimensioni prima di inviare modifiche.

2.4. ADO.NET: Connection Pooling

- Cos'è: Tecnica di ottimizzazione automatica dal client ADO.NET per minimizzare i costi di apertura/chiusura connessioni.
- Perché: Creare una nuova connessione fisica è costoso.
- Come funziona:
- OPEN(): Pooler cerca una connessione disponibile nel pool; se la trova, la restituisce.
- CLOSE(): Pooler restituisce la connessione al pool anziché chiuderla fisicamente.
- Conseguenza: Migliora notevolmente scalabilità e velocità delle applicazioni web.

2.5. Comandi ADO.NET

- **ExecuteScalar:** Esegue comandi SQL che **restituiscono un singolo valore** (es. COUNT, MAX, MIN, SUM). Restituisce un oggetto da convertire.
- **ExecuteNonQuery:** Esegue comandi SQL che **non restituiscono** dati, ma il **numero di righe** interessate (es. INSERT, UPDATE, DELETE, CREATE TABLE).

2.6. Problemi di ADO.NET

- Il Problema dell'Ibridazione (Object-Relational Impedance Mismatch):
- Sviluppatori lavorano con oggetti, database con tabelle.
- Richiede una costante "traduzione" manuale tra i due mondi, ripetitiva e soggetta a errori.
- Il Problema del Boilerplate Code:
- Codice ripetitivo e standardizzato necessario per ogni operazione.
- **5 passaggi non negoziabili per ogni SELECT:**
 1. Dichiarare e Aprire la Connection.
 2. Dichiarare e popolare il Command.
 3. Eseguire il comando e ottenere il DataReader.
 4. Mappare riga per riga i dati (conversione manuale).
 5. Chiudere e chiamare Dispose() per Connection e DataReader.

- Conseguenza: Rallenta lo sviluppo, aumenta il rischio di errori.
- Obiettivo di EF Core: Ridurre questi 5 passaggi a 1: l'istruzione LINQ.

2.7. ADO.NET vs. Entity Framework

- **ADO.NET:**
 - Controllo totale
 - Più codice
 - Meno astrazione
- **Entity Framework:**
 - Meno codice
 - Più astrazione
 - Supporto per LINQ
 - Gestione automatica delle relazioni e migrazioni

3. Entity Framework Core (EF Core)

3.1. Introduzione a EF Core e ORM

- EF Core: Moderno e leggero Object-Relational Mapper (ORM) di Microsoft per .NET.
- Scopo: Lavorare con oggetti .NET nel codice; EF Core gestisce la traduzione per comunicare con il database relazionale.
- In sintesi: Gli sviluppatori lavorano con le classi, EF Core gestisce le query SQL. Non è necessario conoscere ADO.NET o SQL per le operazioni di base.
- Che cosa è un ORM: Tecnica per convertire dati tra sistemi di tipo incompatibili (da database a object-oriented).
- Tre caratteristiche fondamentali:
 1. Mapping: Definisce come il database si "incastra" negli oggetti e viceversa.
 2. Fetching: Recupera dati dal database e materializza oggetti.
 3. Persistenza del grafo: Salva modifiche agli oggetti, generando query SQL.

3.2. Architettura di Entity Framework Classico

- Livello dei Modelli:
 - Conceptual Model: Modello a oggetti (classi C#).
 - Storage Model: Modello del database (tabelle, colonne, chiavi).
 - Mapping: Traduzione tra classi C# e tabelle DB.
- Livello di Esecuzione della Query: (Top-Down)
 1. LINQ to Entities: Query LINQ su DbSet.
 2. Object Services: Converte query LINQ in Command Tree (rappresentazione astratta).
 3. EntityClient Data Provider: Traduce Command Tree in Entity SQL o lo trasmette.
 4. ADO.NET Data Providers: Esegue query SQL native tramite classi ADO.NET.
- Livello di Materializzazione: (Bottom-Up)
 1. DBDataReader: Risultato della query come flusso di dati grezzi.
 2. EntityDataReader: Strato intermedio.
 3. Object Services / IEnumerable: Materializzazione dei dati grezzi in istanze di classi C#.

3.3. Introduzione a Entity Framework Core (Semplificazione)

- Semplificazione: EF Core ha eliminato molti strati intermedi (EntityClient, Entity SQL), focalizzando su DbContext e Provider di Database.
- Caratteristiche: Semplice, estendibile, open source, multiplatforma.
- Funzioni: O/RM, consente operazioni su DB usando oggetti .NET, elimina la necessità della maggior parte del codice di accesso ai dati.
- Supporta molti database providers.

3.4. I Pilastri di un ORM: Data Mapping in EF Core

- Concetto chiave: Data Mapping.
- EF Core gestisce tre tipi di mapping:
- Mapping Entity-Table: Una classe .NET (Entity) corrisponde a una tabella nel database.
- Mapping Property-Column: Una proprietà della classe corrisponde a una colonna nella tabella.
- Mapping Relations: Le proprietà di navigazione tra classi gestiscono le Foreign Keys nel database.

3.5. Componente Chiave: Il DbContext

- Definizione: Il "ponte" o "sessione" tra applicazione e database.
- Ruoli principali:
- Configurazione della Connessione: Contiene stringa di connessione e configurazione del provider.
- Unità di Lavoro (Unit of Work) e Repository: Espone i DbSet (collezioni che rappresentano le tabelle).
- Il Change Tracker:
- Funzionalità centrale del DbContext.
- Traccia ogni modifica apportata alle Entities (oggetti) dopo il caricamento.
- Ogni Entity ha uno stato (Added, Modified, Deleted).
- Quando si chiama SaveChanges(), il Change Tracker genera istruzioni SQL ottimizzate solo per le modifiche e le esegue in una transazione.

3.6. Configurazione del DbContext e Dependency Injection in .NET 8

- Configurazione: Gestita dal costruttore del DbContext tramite DbContextOptions.
- Separa configurazione applicazione da configurazione runtime.
- Esempio costruttore: DbContextOptions<NomeMioDbContext> options.
- Integrazione con Dependency Injection (DI) in .NET 8:
- DbContext registrato come servizio nel contenitore DI.
- Configurazione in Program.cs con AddDbContext<T>().
- Specifica il Provider (es. UseSqlServer()) e la stringa di connessione (normalmente da appsettings.json).
- Ruolo chiave: DI gestisce automaticamente creazione e distruzione (Dispose) del DbContext, garantendo rilascio corretto delle risorse.

3.7. Perché Usare EF Core su .NET 8+?

- Produttività: Riduzione drastica del codice di accesso ai dati.
- Sicurezza del Tipo (Type Safety): Lavoro con oggetti tipizzati in C#, eliminando conversioni manuali.
- Indipendenza dal Database: Cambio database sottostante (es. SQL Server a PostgreSQL) con minimi cambiamenti nel codice tramite i Provider.
- LINQ: Interrogare il database con Language Integrated Query, linguaggio nativo in C# anziché stringhe SQL.

3.8. Approcci di Sviluppo

- Code First:
 - Principio: Si scrivono prima le classi C# (Entities). Il codice è la Source of Truth.
 - Processo: EF Core, tramite Migrazioni, genera database e schema (tabelle, colonne, relazioni). Modifiche al codice generano nuove Migrazioni per aggiornare il database.
- Database First:
 - Principio: Si parte da un database già esistente. Il database è la Source of Truth.
 - Processo: Strumento di reverse engineering (Scaffold-DbContext) analizza lo schema e genera automaticamente classi Entities C# e la classe DbContext.
- Utile con database esistenti non modificabili dal codice.

3.9. Il Modello di Dati: Entità e Proprietà

- Le Entità: POCO e Convenzioni:
- Entità: Semplice classe C# che rappresenta una tabella. Spesso POCO (Plain Old CLR Objects).
- Convenzioni: EF Core deduce schema.
- Nomi di Tabella: Classe al plurale (es. Prodotto → Prodotti).
- Chiave Primaria: Proprietà Id o NomeClassId (es. ProdottoId).
- Tipi di Proprietà:
- Mapping automatico tipi primitivi .NET a tipi SQL (es. string a nvarchar).
- Obbligatorie (Required): Proprietà di tipo valore (int, DateTime) per default. Proprietà di tipo riferimento (string, class) sono facoltative (NULL in SQL) a meno di usare Nullability di .NET 8.
- Tipi Avanzati: Value Converters per tipi specifici (JSON Columns).
- Mappatura delle Relazioni (Navigation Properties):
- Uno-a-Molti (1:N): Es. Cliente ha List<Ordine>, Ordine ha Cliente e ClientId (FK).
- Molti-a-Molti (N:N):
- Introdotto in EF Core 5.0.
- Non richiede creazione esplicita della classe C# per la tabella di unione (Join Table).
- Basta definire una collezione di navigazione in entrambe le entità. EF Core 8+ genererà automaticamente la tabella di unione nel database.
- Lazy Loading vs. Eager Loading:

- Eager Loading (con Include()): Preferito per efficienza, carica dati correlati nella stessa query.
- Lazy Loading: Richiede proxy e DbContext sempre attivo, usato per comodità.
- Configurazione Esplicita (Data Annotations e Fluent API):
- Quando le convenzioni non bastano.

con la lazy viene caricata solo la classe blog e la lista vuota di post, dalla UI quando sarà necessario ci sarà un caricamento per prendere i dati dalla lista di post.

invece la eager carica già anche la classe post insieme al blog quindi sarà già caricata da UI.

però occhio con i grandi dati perchè se noi carichiamo tutto il db con le classi ecc abbiamo problemi di costi e prestazioni.

- Data Annotations: Attributi direttamente su classi/proprietà ([Table("TblClienti")], [Required]). Veloci, ma "inquinano" il modello.
- Fluent API: Metodo più potente e flessibile. Usato in OnModelCreating() nel DbContext (modelBuilder.Entity<Cliente>().ToTable("Clienti");). Mantiene il modello di dominio pulito.

3.10. Code First Migrations

- Ruolo e Necessità: Mantenere sincronizzato il modello C# con lo schema del database (il codice è Source of Truth).
- Obiettivo: Generare automaticamente istruzioni SQL per applicare modifiche al database senza perdere dati.
- Ciclo di Vita:
- Add-Migration [Nome]: Confronta modello C# con ultimo stato, genera file C# con metodi Up() (applica modifica) e Down() (annulla).
- Update-Database [Nome]: Esegue metodi Up() delle migrazioni non applicate.
- EF Core traccia le migrazioni nella tabella __EFMigrationsHistory.
- Gestione di Conflitti e Rollback:
- Non modificare file di migrazione applicati.
- Rollback: Update-Database NomeUltimaMigrazioneValida esegue i metodi Down().
- In team, migrazioni devono essere applicate in ordine cronologico.
- Le Migrazioni SQL (Scripting):
- Per ambienti di produzione con restrizioni di sicurezza.
- Script-Migration: Genera un singolo file SQL con tutte le istruzioni DDL necessarie per i DBA.

3.11. Database First

- Quando si Utilizza: Database esistente è la Source of Truth (es. progetti legacy, DBA gestisce schema).
- Processo di Reverse Engineering (Scaffold-DbContext):
- Comando (CLI o PMC) per implementare Database First.
- Richiede stringa di connessione, provider, cartella di output.
- Genera classi entità POCO e classe DbContext corrispondenti.
- Gestione delle Migrazioni (Evoluzione dello Schema):
- Migrazioni tecnicamente disattivate per default.
- Modifiche allo schema DB avvengono esternamente.
- Rigenerazione: Scaffold-DbContext deve essere rieseguito, sovrascrivendo le classi generate (eventuali modifiche manuali andranno perse).
- Approccio Misto (Tipi Parziali):
- Le classi generate da Scaffold-DbContext sono partial.
- Si possono creare file .partial.cs separati per aggiungere logica di business, metodi o proprietà non mappate senza che vengano sovrascritte.
- Requisiti per Database First: Connessione a DB relazionale (SqlServer, PostgreSQL), DB esistente, progetto .Net 8+ con dipendenze (Microsoft.EntityFrameworkCore, SqlServer, Tools).

3.12. Querying dei Dati (LINQ TO ENTITIES)

- Introduzione a LINQ e Traduzione:
- Metodo standard per interrogare DB con EF Core.
- Scrivere query direttamente in C# su DbSet e IQueryable.
- Ruolo fondamentale di EF Core: Traduce l'espressione LINQ in un'istruzione SQL ottimizzata prima dell'invio al database.
- Esecuzione della Query: IQueryable vs. IEnumerable:
- **IQueryable<T>** (Query Non Eseguita): Lavorando su DbSet e concatenando metodi (Where(), OrderBy(), Select()), la query non è ancora eseguita. EF Core sta costruendo l'albero della query SQL.
- **IEnumerable<T>** (Query Eseguita): La query viene eseguita quando si applica un metodo che forza la materializzazione (ToList(),FirstOrDefault(), foreach). I dati vengono trasferiti dal DB alla memoria.
- Attenzione: AsEnumerable() o filtri su IEnumerable vengono eseguiti Client-Side (in memoria), con gravi problemi di performance se si caricano troppi dati.
- Caricamento Eager (Eager Loading):
- Le proprietà di navigazione non vengono caricate automaticamente.
- Usare Include() per caricare dati correlati nella stessa query SQL (es. context.Clienti.Include(c=>c.Ordini).ToList();).
- ThenInclude() per dati su più livelli (es. Clienti -> Ordini -> DettagliOrdine).
- Strategia di caricamento più efficiente.
- Operazioni CRUD (Create, Read, Update, Delete):
- Gestite tramite DbContext e Change Tracker.

- Create (Inserimento): `context.Add(nuovaEntita)`. Change Tracker marca come Added.
- Update (Aggiornamento): Modificare proprietà di un'entità caricata. Change Tracker rileva e marca come Modified.
- Delete (Cancellazione): `context.Remove(entitaDaCancellare)`. Change Tracker marca come Deleted.
- Tutte le modifiche pendenti fino a `context.SaveChanges()`, che le raggruppa ed esegue l'aggiornamento transazionale.

3.13. Prestazioni e Ottimizzazione delle Query

- Query Non Tracciate (AsNoTracking()):
- Il Change Tracker traccia le entità caricate per default (necessario solo per modifiche).
- Per query di sola lettura, il tracciamento è inutile.
- Usare AsNoTracking() per evitare overhead di memoria e CPU, garantendo un notevole miglioramento della velocità.
- Proiezioni (Selezionare Solo i Dati Necessari):
- Evitare di caricare l'intera entità quando servono pochi campi.
- Usare Select() per selezionare un oggetto anonimo o un DTO contenente solo le colonne necessarie.
- EF Core genera SQL con solo le colonne specificate, riducendo traffico di rete e lavoro del database.
- Esecuzione di SQL Raw e FromSql:
- Per scenari che richiedono SQL puro (stored procedure, funzioni DB non supportate, query complesse).
- Usare FromSqlRaw() (o FromSqlInterpolated()) su un DbSet.
- I dati restituiti devono mapparsi perfettamente alle proprietà dell'entità target.
- Fondamentale: Usare l'interpolazione per evitare SQL Injection con parametri dinamici.
- Indici e Transazioni Esplicite:
- Indici: Assicurarsi che le colonne usate in Where e OrderBy abbiano indici nel DB. Configurabili tramite Fluent API (HasIndex()).

- Transazioni Esplicite: `SaveChanges()` usa implicita. Per raggruppare più operazioni di `DbContext` in una singola transazione, usare `DbContext.Database.BeginTransaction()`.

3.14. Gestione della Concorrenza e Salvataggio Avanzato

- Concorrenza Ottimistica (Optimistic Concurrency):
- Meccanismo per gestire più utenti che modificano lo stesso record contemporaneamente.
- Problema "Last-In Wins": La modifica più recente sovrascrive le precedenti.
- Soluzione: EF Core introduce una Proprietà Concorrente (Concurrency Token) (es. timestamp o `RowVersion`).
- Quando un utente carica il record, carica anche il token.
- Al `SaveChanges()`, EF Core aggiunge alla clausola `WHERE` la condizione che il token nel DB sia ancora lo stesso.
- Se il token non corrisponde (perché un altro utente ha modificato il record), l'aggiornamento fallisce e EF Core solleva una `DbUpdateConcurrencyException`.
- Lo sviluppatore deve gestire questa eccezione.
- Salvataggio Esplicito di Proprietà Specifiche:
- Normalmente, Change Tracker aggiorna tutte le proprietà modificate.

- Per aggiornare solo proprietà specifiche: `context.Attach()` con l'entità e poi `context.Entry(ordine).Property("Stato").IsModified=true;`
- Utile in scenari disconnessi o con DTO.
- Intercettori (Interceptors):
- Potente meccanismo per intercettare operazioni `DbContext` prima o dopo che si verificano.
- Essenziali per logica trasversale (cross-cutting concerns) non legata al business.
- Casi d'uso: Auditing (DataCreazione, UtenteModifica), Query Logging, Soft Delete (UPDATE anziché DELETE).
- Transazioni di Tipo Async:
- In .NET 8, tutte le operazioni di I/O dovrebbero essere asincrone per non bloccare il thread principale.
- EF Core fornisce controparti asincrone: `ToListAsync()`, `FirstOrDefaultAsync()`, `SaveChangesAsync()`, `ExecuteUpdateAsync()`.
- Best practice: Garantisce uso efficiente delle risorse del server.

4. Prossimi Passi e Risorse

- Esperimenti:
- Migrazioni Complesse (Rinomina/Spostamento).
- Creazione di un Intercettore per Auditing automatico.
- Misurare differenza performance tra query Tracciate e Non Tracciate (AsNoTracking).
- Contatti: massimiliano.sorce@msc.com, Massimiliano Sorce (LinkedIn: <https://www.linkedin.com/in/maxupgrade/>)
- Risorse: Microsoft Learn: <https://learn.microsoft.com/en-us/ef/core/>

4.1 Concorrenza Ottimistica

La **Concorrenza Ottimistica** è il meccanismo che Entity Framework Core utilizza per gestire il problema derivante da più utenti che tentano di modificare lo stesso record contemporaneamente.

Problema (Last-In Wins): Il problema tipico si verifica quando l'Utente A carica un record, l'Utente B lo modifica e lo salva, e subito dopo l'Utente A lo modifica e salva. In questo scenario, la modifica di B viene sovrascritta (il modello *Last-In Wins*).

Soluzione (Concurrency Token): EF Core risolve questo problema introducendo una **Proprietà Concorrente** (*Concurrency Token*), che è spesso un campo timestamp o RowVersion nel database.

Meccanismo:

1. Quando l'Utente A carica il record, carica anche il valore di questo *token*.
 2. Al momento del `SaveChanges()`, EF Core aggiunge una clausola **WHERE** alla query UPDATE per condizionare l'aggiornamento solo se il *token* nel database è **ancora lo stesso** che l'Utente A aveva caricato.
 3. Se l'Utente B ha modificato il record nel frattempo, il valore del *token* nel database sarà diverso. La clausola WHERE non troverà righe corrispondenti e l'aggiornamento fallirà.
 4. In caso di fallimento dell'aggiornamento, EF Core solleva una **DbUpdateConcurrencyException**.
- La gestione di questa eccezione ricade sullo sviluppatore, che deve decidere se ricaricare i dati e chiedere all'utente di risolvere il conflitto.

4.2 Salvataggio Esplicito di Proprietà Specifiche

Normalmente, il Change Tracker rileva e aggiorna tutte le proprietà modificate di un'entità. Tuttavia, in alcuni casi, può essere necessario aggiornare **solo proprietà specifiche**, bypassando il tracciamento completo.

Questa tecnica è particolarmente utile in **scenari disconnessi** o quando si utilizzano **DTO** (Data Transfer Objects) per gli aggiornamenti, volendo evitare di caricare l'entità completa prima di salvarla.

È possibile utilizzare il metodo `Attach()` sul `DbContext` con l'entità che contiene solo le modifiche e poi manipolare esplicitamente lo **stato dell'Entità** all'interno dell'Entry del Change Tracker, ad esempio:

```
context.Entry(ordine).Property("Stato").IsModified=true;
```

4.3 Intercettori (Interceptors)

Gli **Intercettori** sono un potente meccanismo introdotto in EF Core per intercettare le operazioni del DbContext *prima* o *dopo* che si verificano. Sono essenziali per l'implementazione della **logica trasversale** (*cross-cutting concerns*) che non è legata alla logica di business dell'entità.

Casi d'uso comuni per gli Intercettori:

- **Auditing:** Intercettare il metodo `SaveChanges()` per popolare automaticamente campi come la data di creazione/modifica e l'utente modificatore su ogni entità modificata, prima che la query venga inviata al database.
- **Query Logging:** Intercettare la query SQL generata da EF Core per registrarla in un sistema di monitoraggio.
- **Soft Delete:** Intercettare le operazioni di DELETE e sostituirle con un UPDATE che imposta un flag `IsDeleted = true`.

Gli intercettori permettono di iniettare questa logica in modo pulito, senza dover "sporcare" l'entità o il DbContext.